Acoustic Testing Tools

Charlie Laub – 10 November 2026

charlie.laub@mail.de

About:
The Acoustic Testing Tools are a set of measurement programs and post-processing apps for collecting and analyzing acoustic measurements to generate frequency domain response data. These tools perform similar functions as programs such as ARTA and meet the core measurement needs for loudspeaker designers/builders. Also included in the Toolset are apps for processing groundplane measurements. All programs are open source, and the source code is downloadable from my GitHub repository.

Contents

Getting Ready - First Steps: 2
Requirements: 2
Installing Python 2
Running the programs: 2
Suggested Workflow 3
Tips for Organizing Your Data: 3
Measurement Tools 4
ProbeAudioDevs.exe: 4
DaDooRunRun.exe 5
DaDooImdImd.exe 9
Python Post-Processing Tools 12
Time2Freq 12
H1_Time2Freq 17
IMDanalysis 19
Fimpulse2CSD 20
Python Data Plotting Utilities: 21
WAVplot: 21
FRDplot 23
BandKplot 24
Groundplane Measurement Post-Processing Tools 26
GPDP 26
ImpAlign 26

Getting Ready - First Steps

Requirements:

All tools have been prepared for and tested on Windows 11 and Ubuntu Linux 22.04 and later. Other Debian based Linuxes that use glibc version 2.35 or later should also work.

Python is required to use the post processing tools, and the modules numpy, pandas, matplotlib, and scipy must be installed. Instructions for installing python under Windows and Linux are provided in the docs directory.

The measurement programs are designed to perform two-channel measurements ONLY. You will need an audio interface that has two input and two output channels. One input channel should be configured to sense the output, either using a loopback cable from one of the outputs, or connected to the loudspeaker terminals to sense the actual stimulus signal applied to the D.U.T. through a passive voltage step-down network.

If you wish to obtain the response SPL in units of dB re 20 uPa @ 2.83V/1m, you will need to know the following information:

- maximum input voltage for each input on your audio interface in Volts,
- amplifier gain in Volts at the output per Volt at the input (when using the loopback connection), or the voltage step down ratio of the passive sensing network that is connected to the loudspeaker terminals directly,
- microphone sensitivity in mV/Pa,
- distance between microphone and loudspeaker in meters, and
- have your microphone calibration data available in a text file.

Installing Python

Python is easy to install and use, both on Windows and Linux. Using python does not mean “programming” or writing or editing code. Python is an interpreter that reads the program file (the program “code”), compiles it on the fly, and then runs it as a program but without saving an executable binary file.
Instructions for installing python under Windows and Linux are provided in the docs directory.

Running the programs

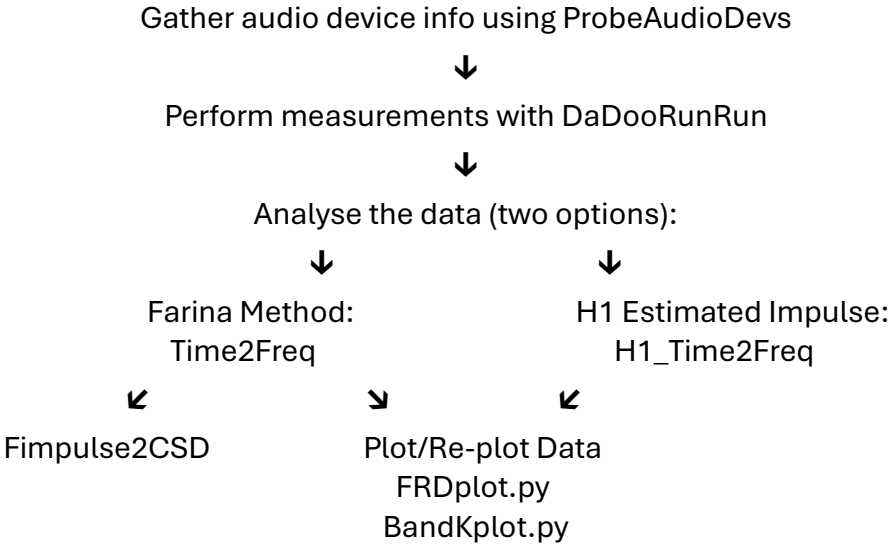
All programs must be run from a terminal and require one or more command line parameters. Under windows, open cmd.exe. A convenient way to open the terminal in any directory is to click in the File Explorer address bar, type cmd, and then press Enter. Under Linux open your preferred command line terminal.

Measurement programs only write information and messages to the terminal and save output files. Most other programs have a GUI but must be launched from the terminal.

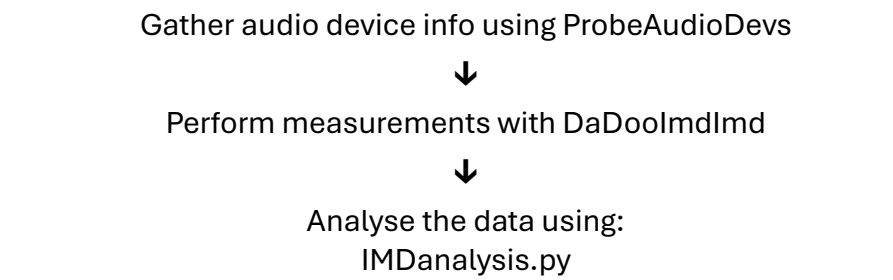
Following the convention used in each OS, Windows programs have the filename extension “exe” (e.g. DaDooRunRun.exe) while the Linux programs lack the filename extension.

Suggested Workflow

There is a natural workflow when using these tools, progressing from measurements to analysis. When your goal is to obtain the frequency response and optionally the harmonic distortion of the loudspeaker:



When your goal is to produce intermodular distortion information about the loudspeaker:



Tips for Organizing Your Data

Before you begin making serious measurements you should stop and plan ahead before jumping in. This includes planning how the data will be organized in files and folders and creating the folder or directory structure before collecting data any. Likewise, planning how you should name the files so that when you come back in a month or a year or more, you can identify the data and recall which conditions or loudspeaker system was being tested.

The programs presented here do not offer the opportunity to leave a trail of notes in the files that you can refer to later, so that is left completely up to you. Use descriptive filenames and directory names. It can be helpful to write a summary text file describing the system, measurement conditions, position of the microphone and loudspeaker, the room or measurement space used, etc.

Measurement Tools

There are three measurement-related tools. These programs only perform boring measurement functions, so there was no point in providing a GUI environment for them. Some informative text messages are written to the terminal during the execution and examples of these are provided below.

The programs were written in C++ and have been compiled and linked into binary executable files. When using Windows, the port-audio DLL must reside in the same directory as the program(s), however, you can move these around your filesystem at will. The port-audio DLL file is supplied in the downloadable zip archive.

ProbeAudioDevs.exe

Use this program first. It allows the user to determine the index of the audio system interface (api_idx) and the input and output device index (device_idx) that will be used by the other measurement programs. Under Linux/ALSA, the same device handles both input and output. Windows assigns different device numbers to inputs and outputs. The device idx values can change after a reboot as new devices are added or removed from the system.

The recommended way to run this program is with the "-c" option. This walks the user through the process of selecting a device. Without the "-c" option the program simply outputs the info for all audio devices on the system, and this can be a lot to review in order to find what you need.

Example usage and output:

```
C:\...\measurement_tools\>ProbeAudioDevs.exe -c

=========================================================
     ProbeAudio - An Audio Hardware Diagnostic Tool
=========================================================

[Interactive Mode] Detected System Driver APIs:
---------------------------------------------------------
 API Index [0] : MME (SYSTEM DEFAULT) (8 devices)
 API Index [1] : Windows DirectSound (8 devices)
 API Index [2] : ASIO <-- [ASIO] (0 devices)
 API Index [3] : Windows WASAPI (6 devices)
 API Index [4] : Windows WDM-KS (6 devices)

Enter API Index to investigate:
```

Above is the terminal output from ProbeAudioDevs.exe run with the -c command line option. In this first step, the user is asked to select the API index to investigate further. As an example, in the next step I have entered "3" to see a list of the WASAPI devices. This produces the following output:

```
Enter API Index to investigate: 3

[Interactive Mode] Grouped Hardware Names under API: Windows WASAPI
---------------------------------------------------------
 [0] VB-Audio Virtual Cable (3 endpoint entries)
 [1] 2- AMD High Definition Audio Device (2 endpoint entries)
 [2] Senary Audio (1 endpoint entries)

Enter Device Name Index to inspect:
```

This is a numbered list of devices by name. Enter the number that corresponds to the device name you wish to explore further to see all of the Windows audio devices available for it:

```
Enter Device Name Index to inspect: 0
```

```
[Results] Showing all 3 endpoint entry/entries for: "VB-Audio Virtual Cable"

-----------------------------------------------------
Device Index [16]: CABLE Input (VB-Audio Virtual Cable)
  * Managed by Driver API  : [3] (Windows WASAPI)
  * Max Input Channels     : 0
  * Max Output Channels    : 2
  * Default Sample Rate    : 96000 Hz
  * Low I/O Latency Line   : In: 0 ms | Out: 2 ms
  * Supported Sample Rates : 96kHz
-----------------------------------------------------

-----------------------------------------------------
Device Index [20]: CABLE In 16ch (VB-Audio Virtual Cable)
  * Managed by Driver API  : [3] (Windows WASAPI)
  * Max Input Channels     : 0
  * Max Output Channels    : 2
  * Default Sample Rate    : 48000 Hz
  * Low I/O Latency Line   : In: 0 ms | Out: 2 ms
  * Supported Sample Rates : 48kHz
-----------------------------------------------------

-----------------------------------------------------
Device Index [21]: CABLE Output (VB-Audio Virtual Cable)
  * Managed by Driver API  : [3] (Windows WASAPI)
  * Max Input Channels     : 2
  * Max Output Channels    : 0
  * Default Sample Rate    : 96000 Hz
  * Low I/O Latency Line   : In: 3 ms | Out: 0 ms
  * Supported Sample Rates : 96kHz
-----------------------------------------------------
Diagnostic run complete.
```

The output above is typical under Windows. Each section corresponds to a different Windows audio device. By inspecting the values corresponding to “Max Input Channels” and “Max Output Channels” and sometimes the descriptive text on the first line of each section, you can tell whether this device can be used for input or output. For Windows, each device will have a single viable sample rate associated with it. Write down or remember the values listed for the API (in this case “3”) and for the device index you wish to use for input and output, for example “16” for input and “21” for output.

Under Linux there will be much less output from ProbeAudioDevs because the same device can be used at various sample rates, and for both input and output. These characteristics will be reflected in the values listed for Max Input Channels, Max Output Channels, and Supported Sample Rates. Under Linux, the value you will use for the input_device_idx and output_device_idx in the data collection programs will be identical.

Once you have determined the correct index for the API and input/output devices that correspond to your audio interface, you are ready to use one of the data collection programs below.

DaDooRunRun.exe

The program DaDooRunRun collects data that will be used to generate frequency domain information using one of the post-processing tools. It is going to “Do” da multiple data collection “Runs” for you, and is named after a terrible ‘70s era Shaun Cassidy song. The program is designed to play a swept sinusoid “chirp” signal a number of times while collecting data. When data collection is complete, the measurement data from each run is cross-correlated with the other runs to find which is the best “anchor” data set. The program also uses the correlation score to determine if any data runs should be thrown out. Using all of the runs that are found acceptable, the program produces several output WAV files.

You need to tell the program what to do, and there are a number of options for that purpose. These can be set or changed from their defaults using command line switches. The following overview can be obtained by calling the program with the "`--help`" option:

```
C:\... \measurement_tools>DaDooRunRun.exe --help

Acoustic Testing Measurement Tool

Usage: DaDooRunRun.exe [options]

Options:
  --basefilename, -b <file>           base file name for output files (default: results)
  --level, -l <float>                 RMS stimulus level in dB <= 0.0 (default: -10.0)
  --sample_rate, -r <int>             Sampling rate in Hz (default: 48000)
  --chirp_rate, -c <float>            Chirp rate in seconds per decade of frequency (default: 2.0)
  --start_freq, -s <float>            Start frequency in Hz (default: 10.0)
  --num_reps, -n <int>                Number of measurement repetitions (default: 1)
  --alignment_window <float>          Cross-Correlation Alignment Window in milliseconds (default: 5.0)
  --warm_up_delay <float>             Length of DAC warm-up delay in seconds (default: 3.0)
  --rt60, -t <float>                  Room decay time in seconds (default: 3.0)
  --mean_wait, -w <float>             Average pause between runs in seconds (default: 3.0)
  --api_idx, -a <int>                 PortAudio host API index
  --input_device_idx, -i <int>        PortAudio hardware input device index
  --output_device_idx, -o <int>       PortAudio hardware output device index
  --input_channels, -I <id1> <id2>    Two physical input channel indices
  --output_channels, -O <id1> <id2>   Two physical output channel indices
  --min_outfiles, -m                  Do not save the Farina inverse filter or Stimulus.
  --help, -h                          Display this help message and exit

NOTES:
    Both output channels produce the same stimulus.
    The first input channel records the stimulus signal.
    The second input channel records the microphone signal.
    Command line parameters may be provided in any order.
```

For most parameters there is both a long and a short option name, e.g. "`--basefilename`" is the long name and "-b" is the short name. If there is a default value, it is listed at right at the end of the option description. When a default value exists, you do not need to supply that option on the command line.

Program Options:

`--basefilename` : the program generates several WAV files. Output files are named using the convention basename + file_identifier + ".wav". You only need to provide the basefilename, and the program appends some preset file identifiers. For example, when the default basefilename "results" is used, the program produces:

```
results_m.wav : microphone measurement data file
results_s.wav : stimulus data file
results_Fif.wav : Farina inverse filter file
results_h1.wav : H1 estimated impulse file
```

A few of the important measurement parameters are also written into the WAV file, so that later only those files that correspond to the same measurement or measurement conditions can be processed together.

`--level` : sets the playback signal level. If your audio interface does not include a volume control, use this parameter to set the output level. The default value is -10 dB.

`--sample_rate` : sets the sample rate. Note that in Windows you must use whatever sample rate that is set or chosen through the operating system or ASIO driver, so set the rate there first. Under Linux DaDooRunRun can itself set the rate to any rate supported by the audio card.

`--chirp_rate` : this sets the rate at which the frequency increases during the sweep.

`--start_freq` : the chirp begins at this frequency. Note that during the first octave the signal level is ramped up using a windowing function.

`--num_reps` : the number of measurement repetitions to perform

`--alignment_window`: the time interval to search when aligning runs post-measurement.

`--warm_up_delay` : when switching the sample rate on some devices, a settling period is needed in order for the device to function properly. This parameter sets the length of this initial period.

`--rt60` : the measurement space has some reverberation, and the RT60 is the length of time for this to decay towards the noise floor by 60dB. When making repeat measurements, you want to include this decay "tail" in the collected data set. Choose a value that matches the actual RT60 of your measurement space.

`--mean_wait` : an additional pause can be added after the data collection for each run has concluded. This wait time is specified using the mean_wait parameter. The actual wait time is varied by 10% from this value.

`--api_idx, --input_device_idx, --output_device_idx` : You obtain the values for these parameters by running the program ProveAudioDevs.exe first and noting the values that correspond to the audio interface or device that you wish to use for your measurements.

`--input_channels first_channel second_channel`: The program is designed to perform two-channel measurements ONLY. This means one input channel will record the stimulus output (e.g. via a loopback cable) and the other will record the voltage output produced by the microphone. These channel numbers are provided as a pair of integers, and it is important that:

THE FIRST CHANNEL LISTED MUST BE THE LOOPBACK CONNECTION

THE SECOND CHANNEL LISTED MUST BE THE MICROPHONE SIGNAL

Here "first" and "second" refers to the order the channels are declared. Exactly which channels you choose on your audio interface to make these connections is up to you, and any channel number can be assigned to be "first" or "second".

For all OSes, it is important to note that:

CHANNEL NUMBERING STARTS WITH CHANNEL 0

For example, if you have a two-input, two-output audio interface the logical channel numbers will be 0 and 1. Using this as an example, let's assume you have used ProbeAudioDevs and identified that your api index is 2, the device index for input is 27 and the device index for output is 34. Using the short option switches you would specify these values as follows:

```
DaDooRunRun.exe -a 2 -i 27 -I 0 1 -o 34 -O 0 1 …
```

where "…" indicates that additional parameters may be included but have been omitted. The example shows how to list channels as a space-separated pair in the order "first_channel second_channel". In the example, the first channel is 0 and the second channel is 1, so "`-I 0 1`" is used. This could also be written using the long parameter version as "`--input_channels 0 1`" but this takes up a lot of space and keystrokes to write out in full for all the parameters you need to supply.

`--output_channels first_channel second_channel`: An identical audio signal is sent to both output channels. Typically, one is used for a loopback connection, and the other is connected to an amplifier and then the loudspeaker. The channels used for output are provided as a pair of integers.

`--min_outfiles`: when this option is supplied, only the measurement data will be saved to file. This is useful when making multiple measurements on the same DUT under nearly identical conditions, e.g. when collecting a set of on/off axis angle measurements for which the stimulus will be identical. In that case there is no need to repeatedly save the stimulus WAV and other files.

Example Program Output:

Output messages about the data collection are printed to the terminal. An example of these is provided below:

```
./DaDooRunRun -t 2 -w 1 -a 0 -i 1 -I 0 1 -o 1 -O 0 1 -n 7

============ Program Parameters ============
      base file name : results
         sample rate : 48000 Hz
          chirp rate : 2 seconds per decade Hz
          start freq : 10 Hz
            num_reps : 7
    alignment window : 5 milliseconds
       warm up delay : 3 seconds
     room decay time : 2 seconds
           mean wait : 1 seconds
               level : -10 dB
             api idx : 0
    input device idx : 1
      input channels : [0, 1]
   output device idx : 1
     output channels : [0, 1]
        min_outfiles : [false]
============================================


Initializing and Validating the Audio Device...

Driver API: ALSA
Input Device: M4: USB Audio (hw:1,0)
Output Device: M4: USB Audio (hw:1,0)
  First input channel number: 0
  Second input channel number: 1
  First output channel number: 0
  Second output channel number: 1

Starting acoustic measurement session at 48000 Hz...
Opening hardware stream with 2 inputs and 2 outputs.
Warming up the duplex hardware stream for 3 seconds...

Probing baseline hardware latency...
Probe cross-correlation peak found at index: 720 (NCC score: 0.977019).
    An I/O Delay of 15 milliseconds (720 samples) was found.

Begin Collecting Data:

Pausing 1.08551 seconds before collecting data.
Begin measurement #1 of 7...
Pausing 1.02551 seconds before collecting data.
Begin measurement #2 of 7...
Pausing 0.966956 seconds before collecting data.
Begin measurement #3 of 7...
Pausing 0.997892 seconds before collecting data.
Begin measurement #4 of 7...
Pausing 1.07677 seconds before collecting data.
Begin measurement #5 of 7...
Pausing 1.0415 seconds before collecting data.
Begin measurement #6 of 7...
Pausing 1.02516 seconds before collecting data.
Begin measurement #7 of 7...

--- Evaluating Measurement Consistency & Selecting Consensus Anchor ---
  --> Selected MEASUREMENT RUN 5 as the consensus reference anchor (Avg Agreement Score: 0.983979).

--- Performing FFT-Optimized Alignment & Quality Control ---
Searching within a window of +/- 5 ms (240 samples at 48000 Hz) for the best correlation with MEASUREMENT RUN 5:

RUN 1 has a timing offset of -143.994 samples and a quality score of 0.998429.
  --> Correcting the timing error for this run. RUN ACCEPTED for further processing.
RUN 2 has a timing offset of -0.00288138 samples and a quality score of 0.907989.
  --> The quality score is below the cutoff (0.95). RUN REJECTED (likely corrupted).
RUN 3 has a timing offset of -0.00361878 samples and a quality score of 0.999374.
  --> RUN ACCEPTED for further processing.
RUN 4 has a timing offset of -0.0017257 samples and a quality score of 0.999372.
  --> RUN ACCEPTED for further processing.
RUN 6 has a timing offset of 0.00435885 samples and a quality score of 0.999385.
  --> RUN ACCEPTED for further processing.
RUN 7 has a timing offset of 0.00365729 samples and a quality score of 0.999327.
```

```
  --> RUN ACCEPTED for further processing.

6 out of 7 reps will now be averaged.

--- Saving Data Files ---
Measurement data WAV file and custom xDAT metadata written to the file results_m.wav
Stimulus WAV file and custom xDAT metadata written to the file results_s.wav
Impulse response WAV file and custom xDAT metadata written to the file results_h1.wav
Farina inverse filter WAV file and custom xDAT metadata written to the file results_Fif.wav

------------ All tasks completed ------------
```

The output begins with a list of all of the program options used and their values. HINT: save this output to a file so that you can refer back to it in the distant future. The next section of the output describes the initialization and performs a test of the system latency. Thereafter the data collection runs are performed. Once the runs have completed, they are compared and one "anchor" run is selected. This run has the highest agreement with the other runs. It is then used to perform a quality analysis on the remaining runs. In this case, the data from run 2 contained a glitch or measurement artifact, so it was thrown out. You can see the results for the other runs in the analysis. Whenever the measurement is thrown out, the corresponding stimulus data set is also discarded. In the end, 6 of the 7 runs were used to obtain an average data set and the data was written to the files, as indicated by the last few lines.

If you look at the reported offsets for each run, you can see that the data acquisition system timing slipped somewhere during run 2, causing the glitch in the data. When run 4 is used as the anchor or reference data set, the run 1 timing is shifted by almost 144 samples. At 48kHz this corresponds to a time slip of exactly 3 milliseconds. These kinds of timing changes are not uncommon when using USB audio interfaces. The program automatically corrects these timing differences to sub-sample accuracy before averaging the data sets.

DaDooImdImd.exe

The program DaDooImdImd.exe is used to collect measurement data that can be further processed to obtain the intermodulation distortion level for the device under test (DUT), e.g. the loudspeaker. This program is related to DaDooRunRun but is for IMD measurement, so a similar silly name was adopted from the Ministry. Rather than a chirp, this program calculates a multi-tone signal comprised of `--ppo` frequencies per octave and then plays it without interruption multiple times. The default parameters are recommended including sample rate, however, the user can change these as they wish. As data collection proceeds, the program checks in real time that each segment is uncorrupted and will re-do a segment in case of a problem until `--num_reps` valid runs have been collected.

The command line parameters can be listed using the "`--help`" option, and are as follows:

```
Multi-Tone IMD Measurement Tool

Usage: DaDooImdImd.exe [options]

Options:
  --basefilename, -b <file>          Base file name (default: imd_results)
  --level, -l <float>                RMS stimulus level in dB <= 0.0 (default: -10.0)
  --sample_rate, -r <int>            Sampling rate in Hz (default: 48000)
  --fft_exp, -e <int>                FFT Size as power of 2 (default: 16 -> 65536)
  --start_freq, -s <float>           Start frequency in Hz (default: 20.0)
  --end_freq, -f <float>             End frequency in Hz (default: 20000.0)
  --ppo, -p <float>                  Tones per octave (default: 3.0)
  --num_reps, -n <int>               Number of valid runs to average (default: 16)
  --warmup_blocks, -w <int>          Blocks to play before recording (default: 2)
  --api_idx, -a <int>                PortAudio host API index
  --input_device_idx, -i <int>       PortAudio hardware input device index
  --input_channels, -I <id1> <id2>   Two physical input channel indices
  --output_device_idx, -o <int>      PortAudio hardware output device index
```

```
  --output_channels, -O <id1> <id2>   Two physical output channel indices
  --help, -h                          Display this help message and exit
```

`--basefilename` : the program generates several WAV files. Output files are named using the convention basename + file_identifier + ".wav". You only need to provide the basefilename, and the program appends some preset file identifiers. The program will output two files, for example using the default basefilename:

```
imd_results_IMD_MIC.wav
imd_results_IMD_STIM.wav
```

The files are used as input by the IMDanalysis post-processing app to generate the intermodular distortion as a function of frequency.

`--level` : sets the playback signal level. If your audio interface does not include a volume control, use this parameter to set the output level.

`--sample_rate` : sets the sample rate. Note that in Windows you must use whatever sample rate that is set or chosen through the operating system or ASIO driver, so set the rate there first. Under Linux DaDooRunRun can itself set the rate to any rate supported by the audio card.

`--fft_exp` : The user can specify the FFT size to use. To restrict the sizes to powers of 2, the user provides the exponent X that gives rise to an FFT size of 2^X. The default value is recommended.

`--start_freq` : the chirp begins at this frequency. During the first octave the signal level is ramped up using a windowing function.

`--ppo` : the number of tones per octave used in the multi-tone stimulus.

`--num_reps` : the number of measurement repetitions to perform.

`--warmup_blocks` : this number of repetitions are run before data collection begins, in order to bring the measurement space to steady-state.

`--api_idx, --input_device_idx, --output_device_idx` : You obtain the values for these parameters by running the program ProveAudioDevs.exe first and noting the values that correspond to the audio interface or device that you wish to use for your measurements.

`--output_channels` and `--output_channels` : These are specified as pairs of integers. See the description of these parameters for DaDooRunRun for complete details.

Example Program Output:
Output messages about the data collection are printed to the terminal. An example of these is provided below:

```
./DaDooImdImd -a 0 -i 1 -I 0 1 -o 1 -O 0 1

============ IMD Parameters ============
      base file name : imd_results
         sample rate : 48000 Hz
        FFT exponent : 16 (65536 bins)
          start freq : 20 Hz
            end freq : 20000 Hz
   points per octave : 3
            num_reps : 16
       warmup blocks : 2
               level : -10 dB
========================================


Generating multi-tone array with 30 perfectly aligned tones...
```

```
Optimized Crest Factor: 11.4012 dB

Starting Continuous IMD Data Collection...
Averaging Block: 15 / 16

Data Collection Complete.
Saved TSA Mic Data to: imd_results_IMD_MIC.wav
Saved Ideal Stimulus to: imd_results_IMD_STIM.wav
```

There is significantly less output compared to DaDooRunRun. In this case, all 16 runs were performed without a problem. If a run is rejected, a message will report the event.

Sometimes, when first running DaDooImdImd all runs are rejected. Because the program repeats rejected runs, it will be stuck in a sort of loop as it keeps trying to record uncorrupted data. The QA check in DaDooImdImd is less sophisticated, and the first run is always chosen as the anchor. If the first run gets corrupted, all subsequent runs will be rejected and the program cannot complete successfully. If this happens, kill the program using control-c and run it again.

Also, under Linux, there may be several lines reporting that services like Jack cannot be initialized or opened. These can be ignored.

Python Post-Processing Tools

Apart from the measurement tools, all other tools are python apps that come as source code that is run much like a batch script. For your amusement, most include a graphical user interface. You run these programs from the terminal like this:

```
py/python3 program_name.py program_option1 [other_program_options]
```

You type “py” (Windows) or python3 (Linux) to invoke the Python interpreter and then provide the name of the program “program_name.py” plus one or more command line options. Python compiles and builds the code on the fly into a running program and then launches it.

Time2Freq

Time2Freq is a program that implements the Farina method to obtain frequency domain information from the time domain measurement data generated with DaDooRunRun. The program has a graphical user interface but must be invoked/run from the command line. The program will produce the frequency and harmonic distortion responses and save these to FRD files. The program also creates several useful plots and saves them as png files.

There are several command line parameters (use the “`--help`” option to list these):

```
C:\...\post_processing_tools>py Time2Freq.py --help

usage: Time2Freq.py [-h] -b BASENAME_INFILES [-B BASENAME_OUTFILES] [-c MIC_CAL_FILE] [--AES17_SPL] [-k
SPL_CORRECTIONS] [--time_outfile TIME_OUTFILE]

About Time2Freq: An interactive program to process loudspeaker measurements collected using the program
DaDooRunRun. Exports frequency domain response data and plots.

options:
  -h, --help            show this help message and exit
  -b, --basename_infiles BASENAME_INFILES
                        Base file name for the input WAV files.
  -B, --basename_outfiles BASENAME_OUTFILES
                        Base file name for output files (default: response).
  -c, --mic_cal_file MIC_CAL_FILE
                        Path to microphone calibration file (*.mic or *.cal).
  --AES17_SPL           If set, use the AES17 SPL level convention.
   -v, --SPL_at_drive_voltage
                        If set, report SPL at the actual voltage applied to the loduspeaker during the measurement.
                        When left unset, report the SPL normalized to an applied voltage of 2.83 Volts.
   -k, --spl_corrections SPL_CORRECTIONS
                        Comma-delimited quoted string of the five parameters used to set the SPL level.

                        Example: -k "V1=6.15, V2=2.45, Ag=20.0, Smic=10.0, d=0.25"

                        V1   : Channel 1 (loopback) input sensitivity, Volts to reach 0dBFS (+/-1.0) on the audio
                               interface
                        V2   : Channel 2 (microphone) input sensitivity, Volts to reach 0dBFS (+/-1.0) on the audio
                               interface
                        Ag   : Amplifier voltage gain, Volts at speaker terminals per Volt applied to the input
                        Smic : Microphone sensitivity in mV/Pa
                        d    : The distance in meters between the loudspeaker and microphone during the
                               measurement.
  --time_outfile TIME_OUTFILE
                        When supplied, the time domain response will be saved with this file name in the WAV
                               format.
```

Time2Freq reads multiple WAV input files. The option `--basename_infiles` should correspond to the basefilename that was used in DaDooRunRun when performing the measurements and generating the files.

`--mic_cal_file`: provide the file path name to the file containing the mic calibration data. This file contains two columns, frequency in Hz and the level in Decibels. Any lines beginning with the characters '*', '#', or ';' are considered comment lines and will be ignored.

`--AES17_SPL`: When this option is present, it reports the SPL levels following AES17 specification where a peak internal signal level value of 1.0 is reported as 0dB. When measuring loudspeakers you want the RMS level to be reported, so omit the AES17_SPL flag.

`--SPL_at_drive_voltage` : When you would like the SPL data to be calculated for the actual drive voltage that was used during the measurement and reported at that level instead of at the standard voltage of 2.83V, enable this option. This could be used when comparing the frequency responses for different drive voltages, or anytime that normalization to an applied voltage of 2.83V is not desired.

`-- spl_corrections`: This is a text string in double quotation marks, containing a list of five parameters used to set the correct SPL level: These include:

V1, V2: The maximum RMS voltages that produce 0dBFS in your audio interface. This spec might be listed by the manufacturer as “maximum input level” and may be different for each input.

Ag: the amplifier voltage gain. This is Volts on the outputs per Volt on the input. Often the amplifier gain is given in Decibels, and that should be converted to a voltage gain.

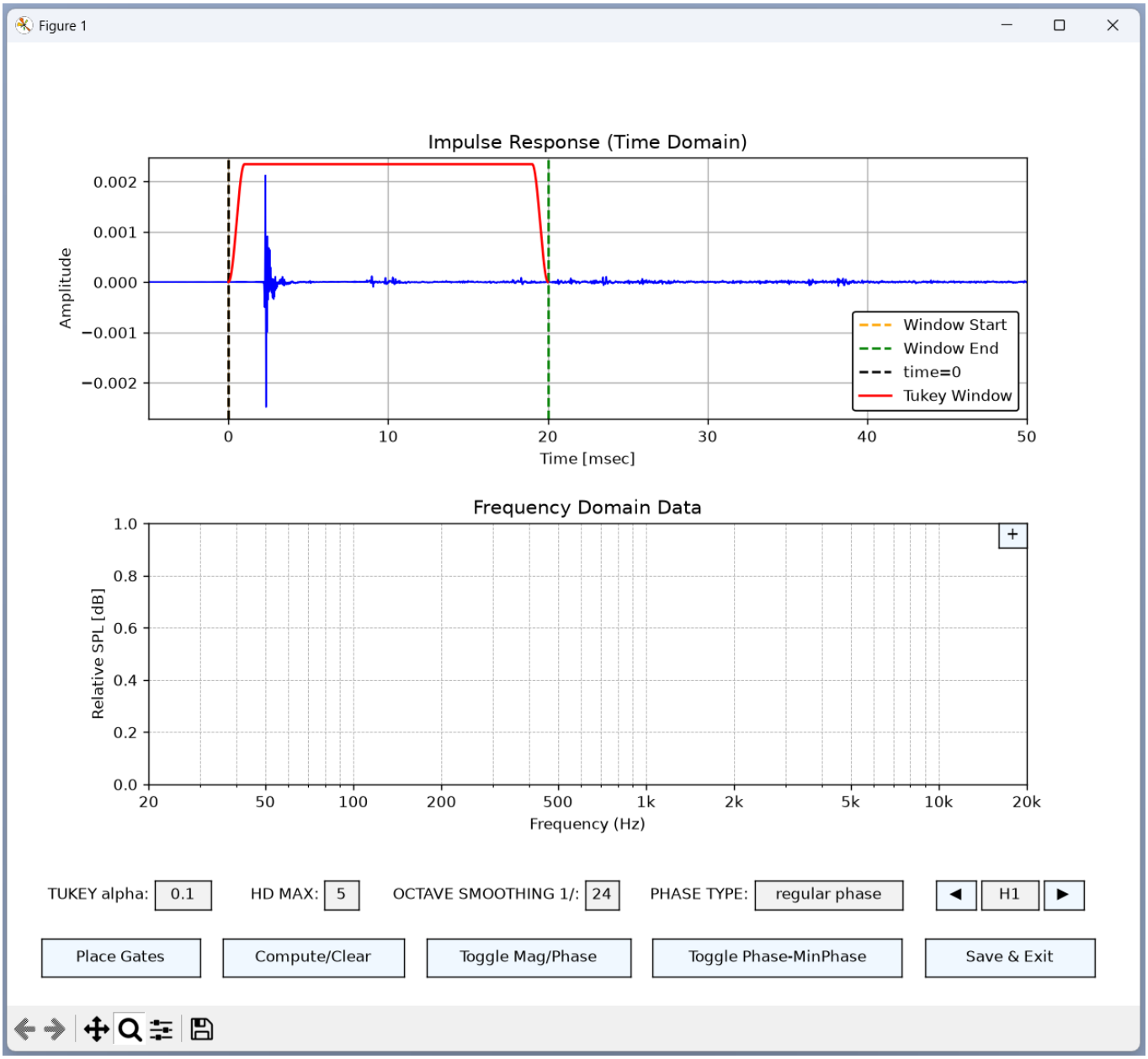
Smic: the microphone pressure sensitivity in mV/Pa. Values are often around 10-50 mV/Pa.

D: the distance between microphone and loudspeaker during the measurement.

`--time_outfile`: This file can be used to create CSD waterfall plots, if desired, using the Fimpulse2CSD.py app.

If the user wishes to calculate the SPL in units of dB re 20 uPa @ 2.83V/1m the microphone calibration file (mic_cal_file) and all the parameters in “spl_corrections” must be provided.

The Time2Freq GUI Interface:



Reminder: The very first action by the user should be to use the zoom tool to bring the portion of the time record into view that contains the direct sound and first reflection.

Controls and Buttons:
TukeyAlpha : sets the width of the window tapers as a fraction of the total window width

HD max : sets the maximum distortion order to calculate

OCTAVE SMOOTHING : sets the points per octave used to smooth the data. The frequency interval set with this control is used both in the plots and also in the FRD data files for each harmonic.

PHASE TYPE : indicates whether regular or minimum phase is selected

◄ H1 ► : the arrows move forward and backwards through the calculated data. The current data set name is shown between the arrows. Hx indicates harmonic number X with 1 being the fundamental. “All” shows a plot of harmonics H1 through HHDmax and THD shows the total harmonic distortion in percent.

Place Gates : this button allows the user to set the window start and end gates (positions). The user must click in the upper impulse response plot twice. The first click sets the time where the start or left side window taper finishes ramping up. Choose a point just where the impulse leaves

the baseline. The second click sets the timepoint where the right or end taper finishes ramping down.

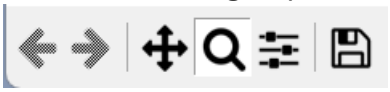
Compute/Clear : computes or clears the frequency and harmonic data.

Toggle Mag/Phase : switches the lower plot between magnitude and phase data

Toggle Phase-MinPhase : switches between regular or minimum phase

Save & Exit : Saves data and plots, then closes the program.

There are also a group of controls in the bottom left corner:



The middle two, the crossed arrows and the magnifying glass, will be used frequently. When you open the app, the zoom control (the magnifying glass) is activated. This should be immediately used to zoom the upper time plot to the portion of the impulse that contains the direct sound up until and just after where the first room or boundary reflection occurs. This is because you cannot easily change the view of the plot while you are placing the gates, so you need to do this first before proceeding to use the “Place Gates” button. The crossed arrows icon activates and deactivates the pan control. This lets you slide the data in the plot left, right, up, and down. You select these controls by clicking on them with the mouse. If you do not like a new view obtained using zoom or pan you can go back to the previous view using the “back” arrow.

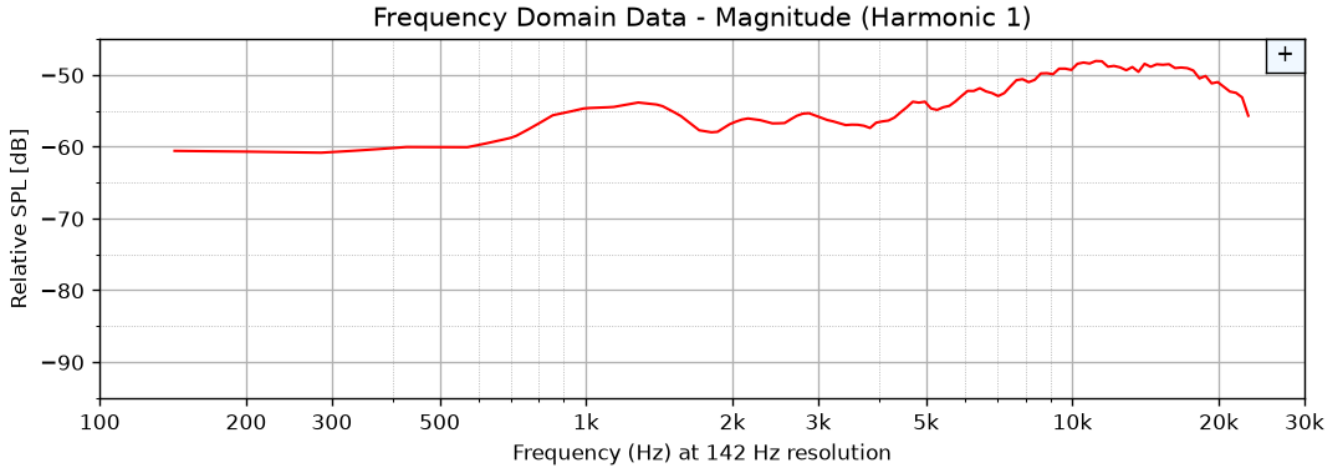
How To Calculate the Frequency Response

To calculate the frequency response you:

1. Zoom the time domain response (upper plot) to show the direct sound and the time point where the first reflection occurs
2. Click the “Place Gates” button and place the gates
3. Click “Compute/Clear” to generate the frequency domain data

Once data is shown you can toggle through the harmonics and ALL/THD plots using the ◀ H1 ▶ control arrows, change to the phase view, etc. If you change the values of HDmax or SMOOTHING, you must clear and re-compute the frequency response data for the new values to take effect.

Example Plots:

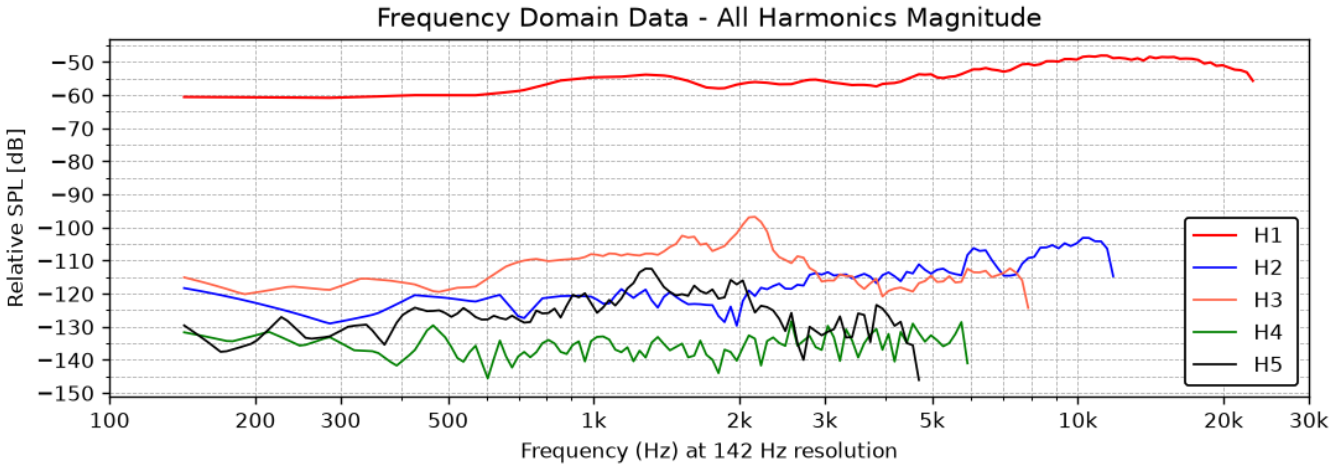


The above is an example of the frequency domain plot for H1 that will appear in the lower window. The data is initially plotted with a 50dB y-axis range. In this case, none of the SPL calibration values were supplied so the SPL values are only relative.

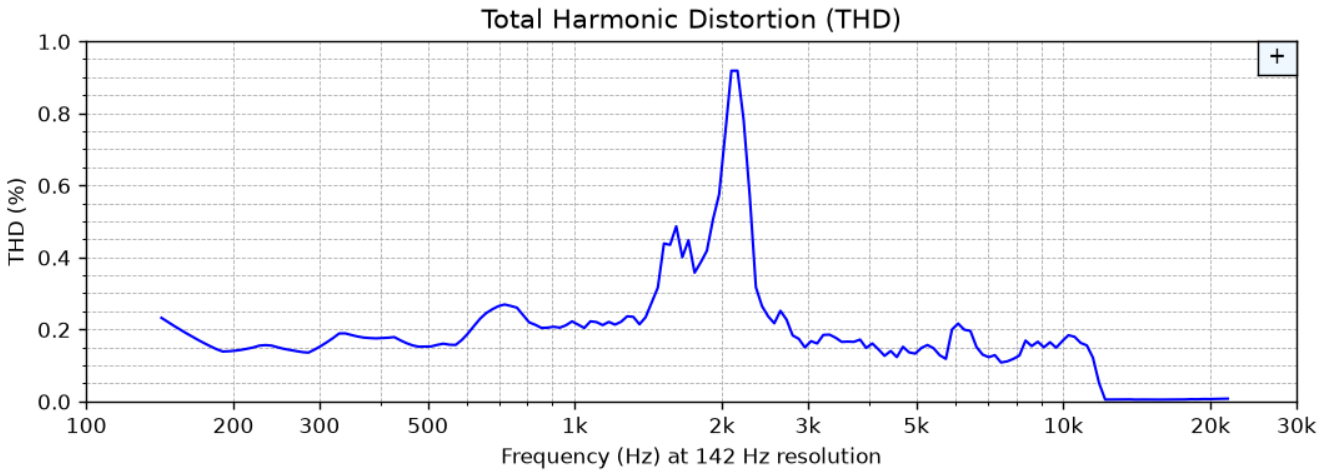
In the upper right hand corner of the plot is a button with a “+” inside. When this is clicked, the axis limits are saved. Thereafter, when returning to this data set, it will be shown again with those axis

limits, otherwise the default 50dB range will be used. The saved axis limits will also be used for the plot of the fundamental frequency response that is saved to a png file upon exit.

It should be noted that the data begins (in this case) around 150Hz. You can see in the label for the x-axis that the resolution of the data is 142 Hz. Data below this frequency is not produced, not in the plot and also not saved to the FRD data files. The data below this frequency is invalid, so it has been eliminated. If you want more low frequency data, you must use a longer time window and this typically means you must make your measurements farther away from reflecting surfaces.



The above plot is an example of the “ALL” plot, showing all the harmonic data on the same plot. The plot will be saved to file. There is no custom axis range setting available since the default view is usually sufficient.



Above is an example of the THD plot. This includes the feature for setting custom axis limits using the “+” button in the upper right hand corner. This plot is saved to a png file upon exit.

Exiting the Program with and without Saving Files

If you wish to stop and exit the program at any time, click the kill button at the top right hand corner of the window. It looks like this: ☒. Clicking the kill button will simply exit the program without performing any other function or action.

If you wish to save all the data and plots and then exit the program, click the “Save & Exit” button instead. Several files will be produced. Each will have a filename starting with the value supplied after the command line parameter “`--basename_outfiles`” (or -B).

If you wish to replot one or more of the FRD files, alone or together, you can use the FRDplot.py app that is also found in the post_processing_tools directory and described later in this document.

H1_Time2Freq

This program was developed before Time2Freq and the interface is similar but has fewer controls. H1_Time2Freq uses the H1 estimated impulse response produced by DaDooRunRun instead of performing the Farina method when generating the frequency domain response.

There are several command line parameters:

```
C:\...\post_processing_tools>py H1_Time2Freq.py --help

usage: H1_Time2Freq.py [-h] [-o OUTPUT] [-s SMOOTHING] [-c MIC_CAL_FILE] [-g END_GATE_TIME] [-a TUKEY_ALPHA] [-m]
                       [--AES17_SPL] [-v] [-k SPL_CORRECTIONS] input_wav

Process loudspeaker impulse response WAV file and export to FRD format.

positional arguments:
  input_wav             Path to the input impulse response WAV file.

options:
  -h, --help            show this help message and exit
  -o, --output OUTPUT   File and pathname to save the output in FRD format (default: FR.frd).
  -s, --smoothing SMOOTHING
                        Response Smoothing in Points per Octave, valid entries are 1..48 (default: 24).
  -c, --mic_cal_file MIC_CAL_FILE
                        Path to microphone calibration file (*.mic or *.cal).
  -g, --end_gate_time END_GATE_TIME
                        Automated mode trigger: End gate time in milliseconds. If provided, the script runs
                          headless without launching the GUI.
  -a, --tukey_alpha TUKEY_ALPHA
                        Tukey window alpha value (default: 0.1).
  -m, --min_phase       Sets the default phase type to minimum phase.
  --AES17_SPL           If set, use the AES17 SPL level convention.
  -v, --SPL_at_drive_voltage
                        If set, report SPL at the actual voltage applied to the loduspeaker during the measurement.
                        When left unset, report the SPL normalized to an applied voltage of 2.83 Volts.
  -k, --spl_corrections SPL_CORRECTIONS
                        Comma-delimited quoted string of the five parameters used to set the SPL level.

                        Example: -k "V1=4.89, V2=2.45, Ag=20.0, Smic=10.0, d=1.5"

                        V1   : (loopback) Volts that produce 0dBFS for this input channel
                        V2   : (microphone) Volts that produce 0dBFS for this input channel
                        Ag   : Amplifier voltage gain, Volts at speaker terminals per Volt applied to the input
                        Smic : Microphone sensitivity in mV/Pa
                        d    : The distance in meters between the loudspeaker and microphone during the
                          measurement.
```

Program Usage:

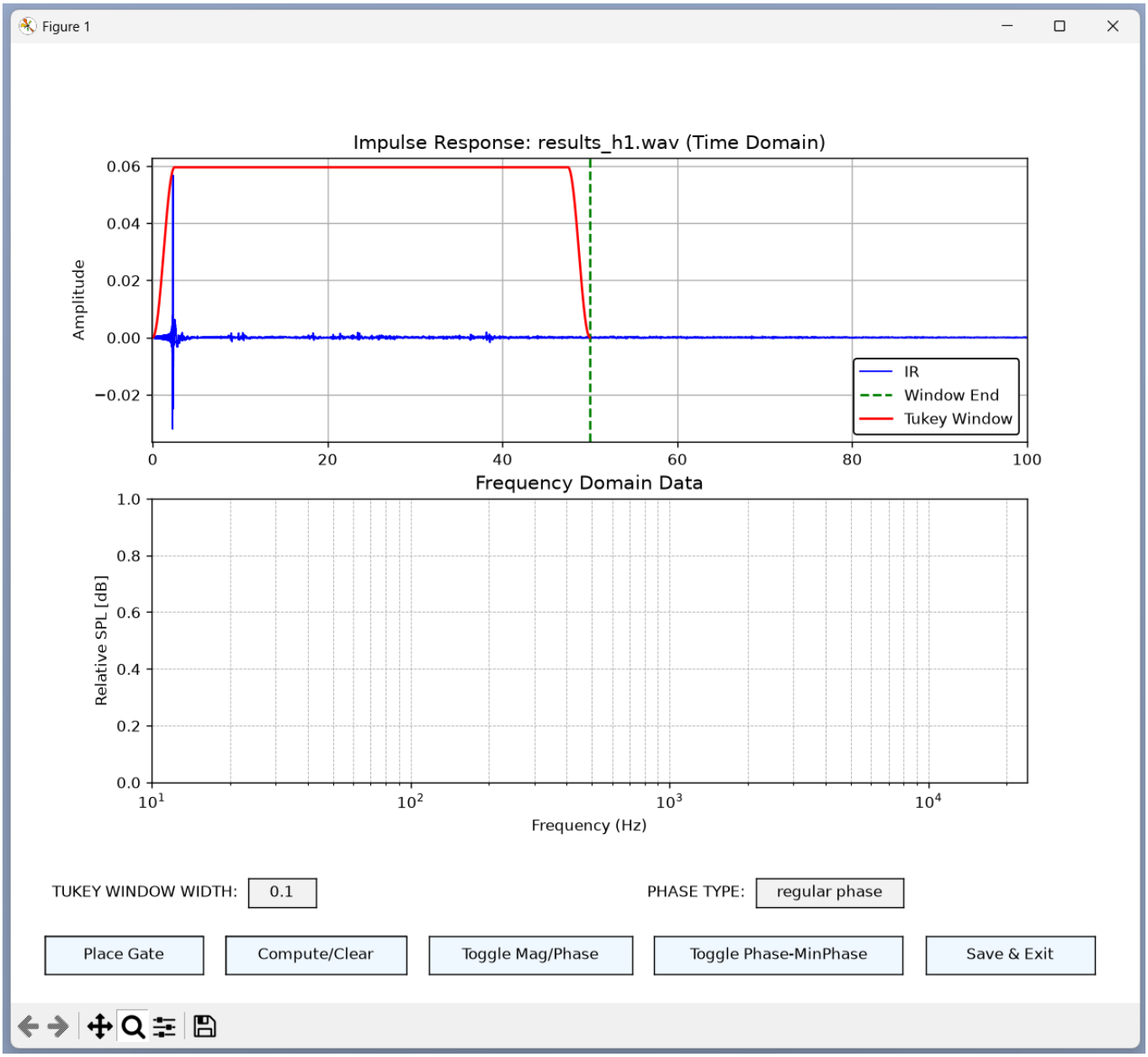
Note that the impulse file is provided without any command line switch. This file is generated by DaDooRunRun and given the "_h1.wav" suffix, e.g. "results_h1.wav". The order of appearance of the options and input filename are arbitrary. An example of how to invoke the program is:

```
C:\...\post_processing_tools>py H1_Time2Freq.py results_h1.wav -c cal_file.mic -k "V1=4.89, V2=2.45, Ag=20.0,
Smic=10.0, d=1.5"
```

To calculate the frequency response, the user begins by choosing only the end gate for the window, with the start of the window always constrained to time=0 so that the correct phase response will be generated. When the Compute/Clear button is clicked, the response is generated and plotted to the lower plot in the window.

Clicking the "Save & Exit" button writes the frequency and phase data to a file in the FRD format with the name specified by the `--output` switch, e.g. FR.frd. The plot is not saved.

The H1_Time2Freq GUI Interface:



Reminder: The very first action by the user should be to use the zoom tool to bring the portion of the time record into view that contains the direct sound and first reflection.

Above is a screen shot of the main window from H1_Time2Freq.py. Many of the controls serve an identical function to those in Time2Freq. This program only calculates the frequency and phase responses and does not produce any distortion information. The user only needs to set the window end gate and then click the Compute/Clear button.

There are some minor differences between the frequency domain response produced via the H1 estimated impulse response and via the Farina method. These are described in the pdf file “AES17_Farina_Scaling.pdf” that can be found in the documents folder of AcousticTestingTools.

Automated Mode:
The primary difference and advantage of H1_Time2Freq versus Time2Freq is that H1_Time2Freq can be run in an automated mode. This is triggered anytime you supply the command line option “end_gate_time”. The motivation for automated processing is, for example, that you are measuring a loudspeaker that remains in a fixed location in the measurement space but is rotated to produce measurements at many angles. For each measurement, the gate time will remain the same and they can be processed using a script that calls H1_Time2Freq repeatedly while changing the input and output filename string. This can save a lot of time and effort compared to firing up the GUI each

time and using the mouse to click on the time plot to set the gate and will ensure that the exact same end gate time will be used for each dataset. Anytime you know in advance the appropriate gate time for the window and do not need to produce distortion information, using H1_Time2Freq will be simpler and easier.

IMDanalysis

The program IMDanalysis.py calculates the intermodular distortion response using the data that was produced by the program DaDooImdImd. The IMDanalysis program has very few command line options, but most are required:

```
C:\...\post_processing_tools>py IMDanalysis.py --help

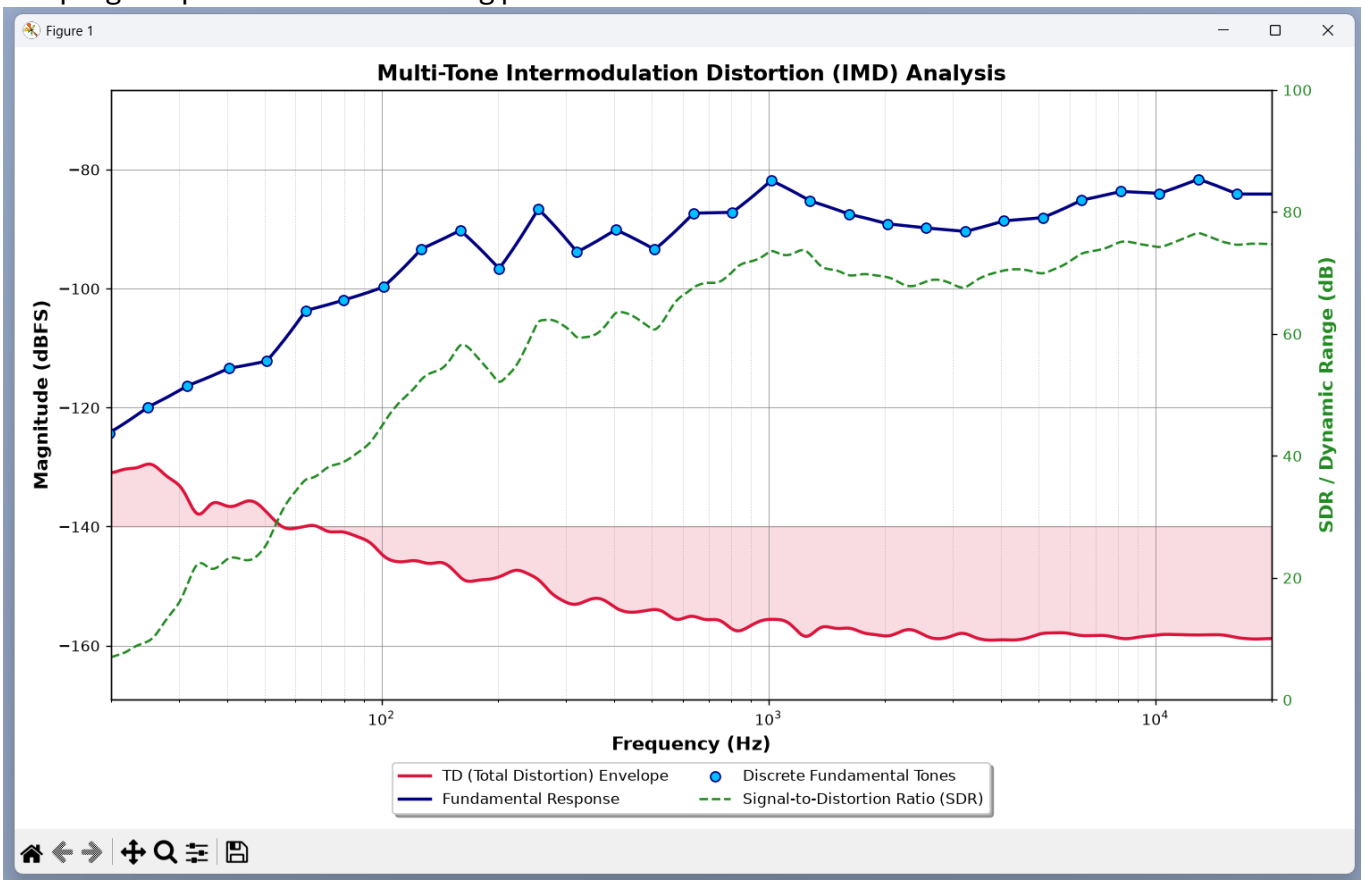
usage: IMDanalysis.py [-h] --stim STIM --mic MIC --out OUT [--smooth SMOOTH]

Process Multi-Tone IMD Measurement Data

options:
  -h, --help       show this help message and exit
  --stim STIM      Path to the Ideal Stimulus WAV file
  --mic MIC        Path to the TSA Measurement WAV file
  --out OUT        Base name for the output files (CSV and PNG)
  --smooth SMOOTH  Smoothing intensity for the distortion hash envelope (default: 8.0)
```

Supply the file path-name for the STIM and MIC files generated by DaDooImdImd and a basename for the output files (if one different than the default is desired).

The program produces the following plot:



The blue line connecting the blue circle data points is the frequency response resulting from the applied signal, with the circles representing the frequencies chosen for the multi-tone stimulus in

DaDooImdImd. The values for the blue line and dots are read off of the primary (left side) y-axis. The red line is the measured IMD level, also read from the primary y-axis. The dashed green line is the ratio of the level of the fundamental to the level of the total intermodular distortion, in Decibels. This is read off of the secondary (right side) y-axis.

The plot and data are automatically saved to file using the output base filename that was supplied on the command line.

There are no user controls. Click the kill button ☒ to exit the program.

Fimpulse2CSD

This app uses the time domain impulse response produced by the Farina method to compute and plot the cumulative spectral decay. The data can be plotted as wireframe surfaces or as heatmaps. Both the CSD and the period normalized CSD (PCSD) are presented, side by side, in the window.

Usage:

```
post_processing_tools>py Fimpulse2CSD.py -help

usage: Fimpulse2CSD.py [-h] filename

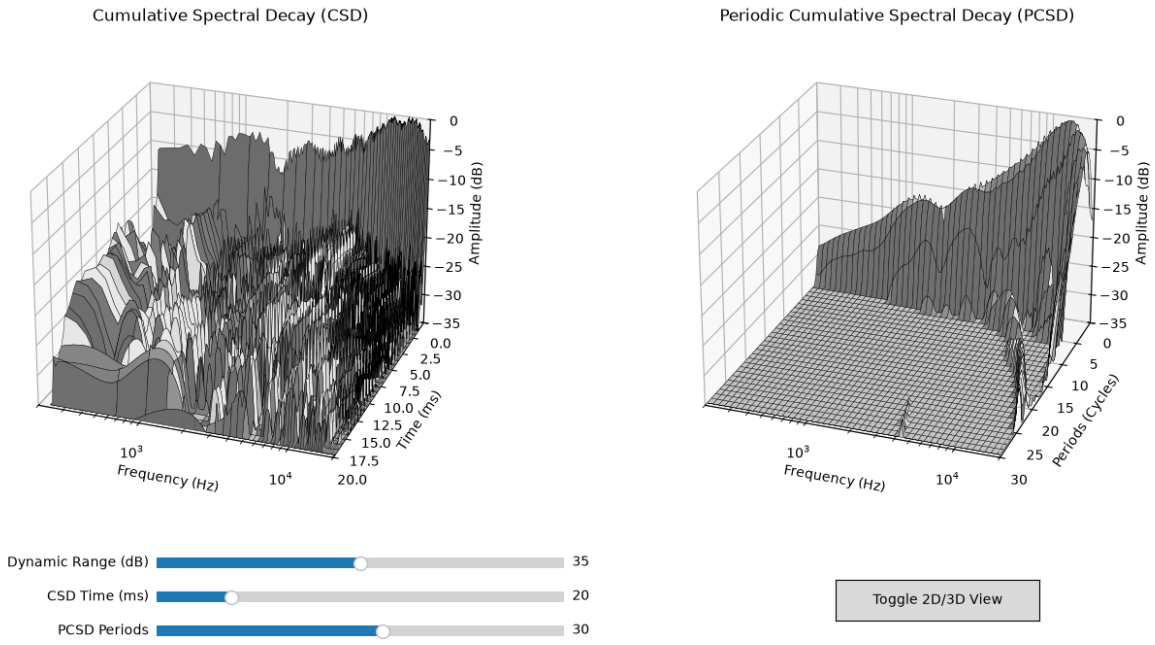
Plot Cumulative Spectral Decay CSD and PCSD in wireframe and heatmap formats.

positional arguments:
  filename    File path-name to the WAV data produced by Time2Freq (via the --time_outfile option).

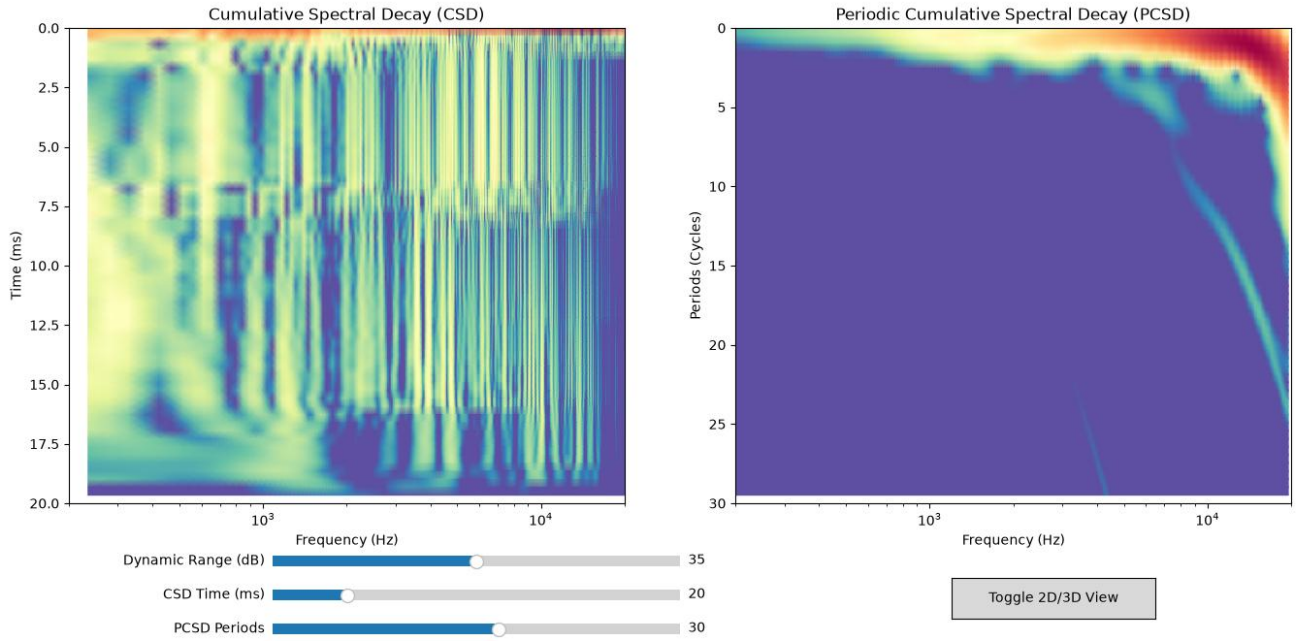
options:
  -h, --help  show this help message and exit
```

There is only one command line option, which should be the name of a WAV file produced by the Time2Freq app using the option "`--time_outfile`".

The wireframe presentation looks like this:



The heatmap presentation is obtained by clicking on the Toggle 2D/3D View button:



There are three controls at bottom left. These control the plot limits and are best set when viewing the wireframe presentation. The dynamic range adjusts the range of the amplitude axis. The maximum value is preset, so this control changes where the floor is placed. The CSD Time and PCSD periods controls adjust the time axis for each plot. This can be set depending on the type of device that is being measured and is limited by the time extent of direct sound in the impulse response. There is no windowing applied, so reflections are included and will contaminate the results. For example, the curved line in the PCSD wireframe and heatmap shows the existence of such a reflection in the data.

To save the plot to a file, click the floppy disc icon:



When you are finished, click the window kill button ☒ to exit the program.

Python Data Plotting Utilities

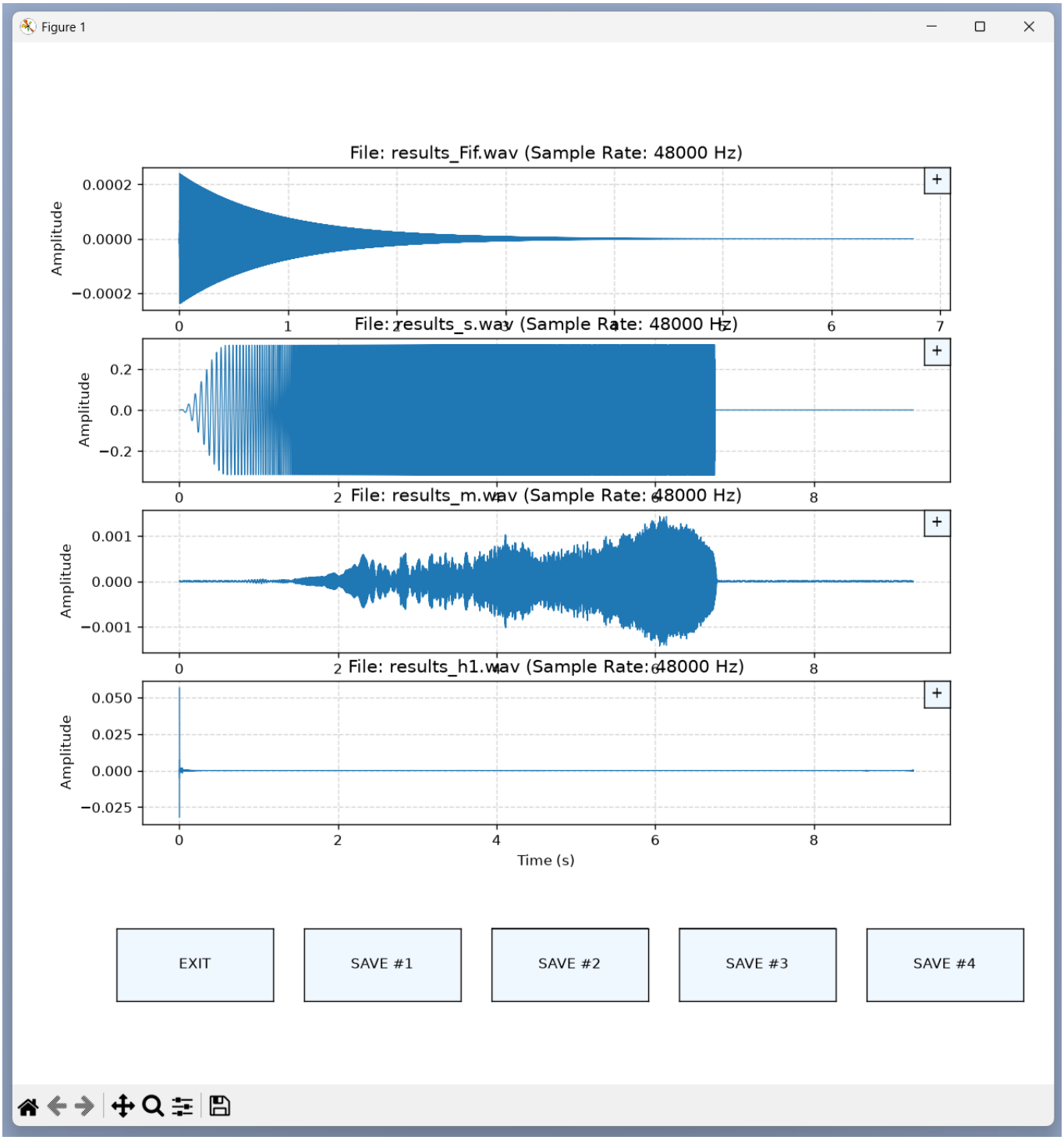
The plotting utilities allow the user to view WAV and FRD data files outside of the post-processing apps, anytime. Each has a slightly different user interface.

WAVplot

The WAVplot.py app can simultaneous plot up to four WAV files, each in their own plot panel. You invoke the program and provide alist of the file(s) you wish to plot. For example:

```
C:\...\post_processing_tools>py WAVplot.py results_Fif.wav results_s.wav results_m.wav results_h1.wav
```

The interface looks like this:



Above, WAVplot is shown plotting all four output files generated by DaDooRunRun. The plots can be manipulated using the python plot controls at the bottom left corner of the window.

There is only one command line flag: "`--sync_plots`". When this Boolean option is set, zooming in one plot causes all other plots to take on the same axis limits. This is useful when you want to compare a small time region between two files. You only need to zoom one plot and the exact same time interval will be shown in the other(s).

Clicking on one of the SAVE buttons will save only that plot panel to a png file at 600dpi. You can save the entire window (showing all plots) to one file by using the floppy disc icon within the group of controls at bottom left. This will also save the buttons to the file.

When --sync_plots is not used, each plot can be zoomed independent of the others. The "+" button has the same function as in other programs, to save the plot limits and re-use these when creating the png plot that is saved to file.

To exit the program, click the EXIT button or the windows' kill button.

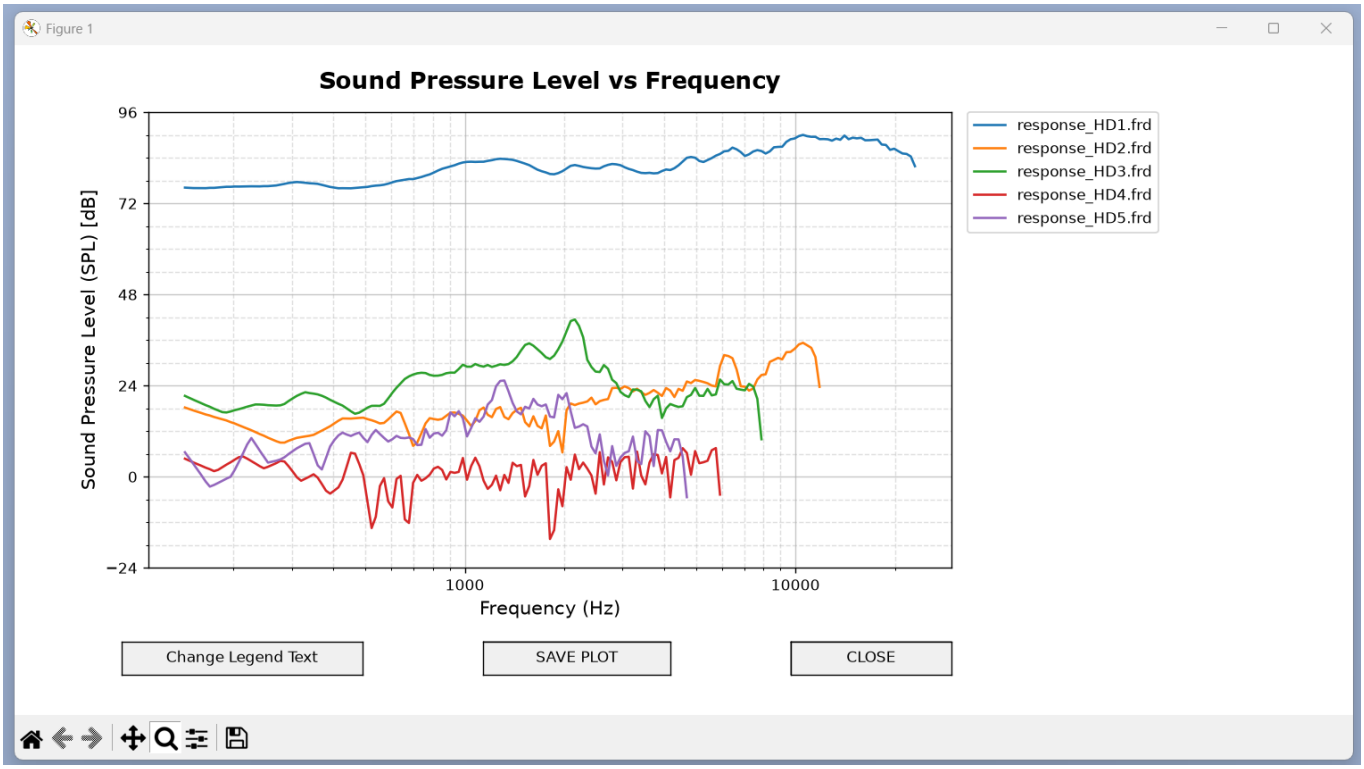
FRDplot

The FRD plot program is used to plot the frequency response data from one or more FRD files in/on the same plot. This can be used, for example, to re-create the fundamental + harmonics plot that is produced in Time2Freq. There are no command line options. The program is simply invoked and a list of FRD files is given, for example:

```
py FRDplot.py my_data1.frd my_data2.frd some_other_data.frd ...etc.
```

There is no limit to the number of FRD files that can be plotted.

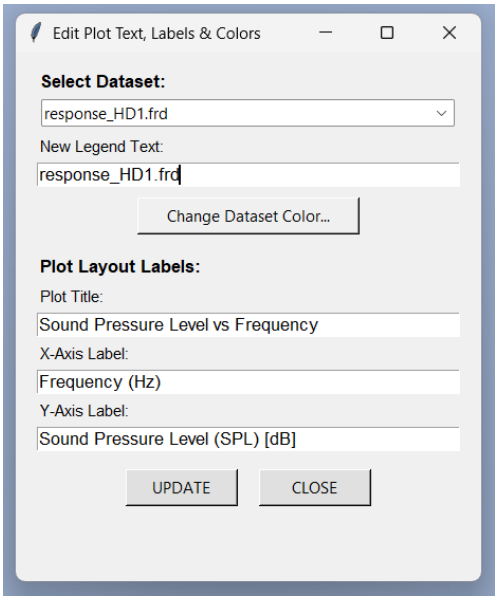
The resulting plot looks like this:



In the plot above I have re-plotted the fundamental and harmonics as an example.

You can zoom and pan this plot and then save that view to a png file using the SAVE PLOT button.

When the plot is first created, the legend contains the filenames. This is not always how one would like the data to be labelled, so there is a control to change the label names and the colour for each line. This tool is launched using the "Change Legend Text" button, and look like this:



You select the dataset that you wish to modify using the drop-down dialog box at the top. Enter the new legend text in the text field just below. Click the UPDATE button at the bottom to change the text. Similarly, choose a new color for the line used for this dataset by clicking on the “Change Dataset Color” button. The plot axis labels and title can also be changed in the text boxes for that purpose. When you are satisfied, click the CLOSE button to close the tool.

BandKplot

Sometimes there is an industry standard way of presenting data that becomes ingrained over time, and it can be helpful to view your data from that perspective. The BandKplot app attempts to perform this function for plots of frequency response. The plot is made with a fixed SPL range of 50dB and with a certain aspect ratio. This echoes the original format produced on early B&K measurement equipment.

One FRD or file can be supplied as an argument to the program:

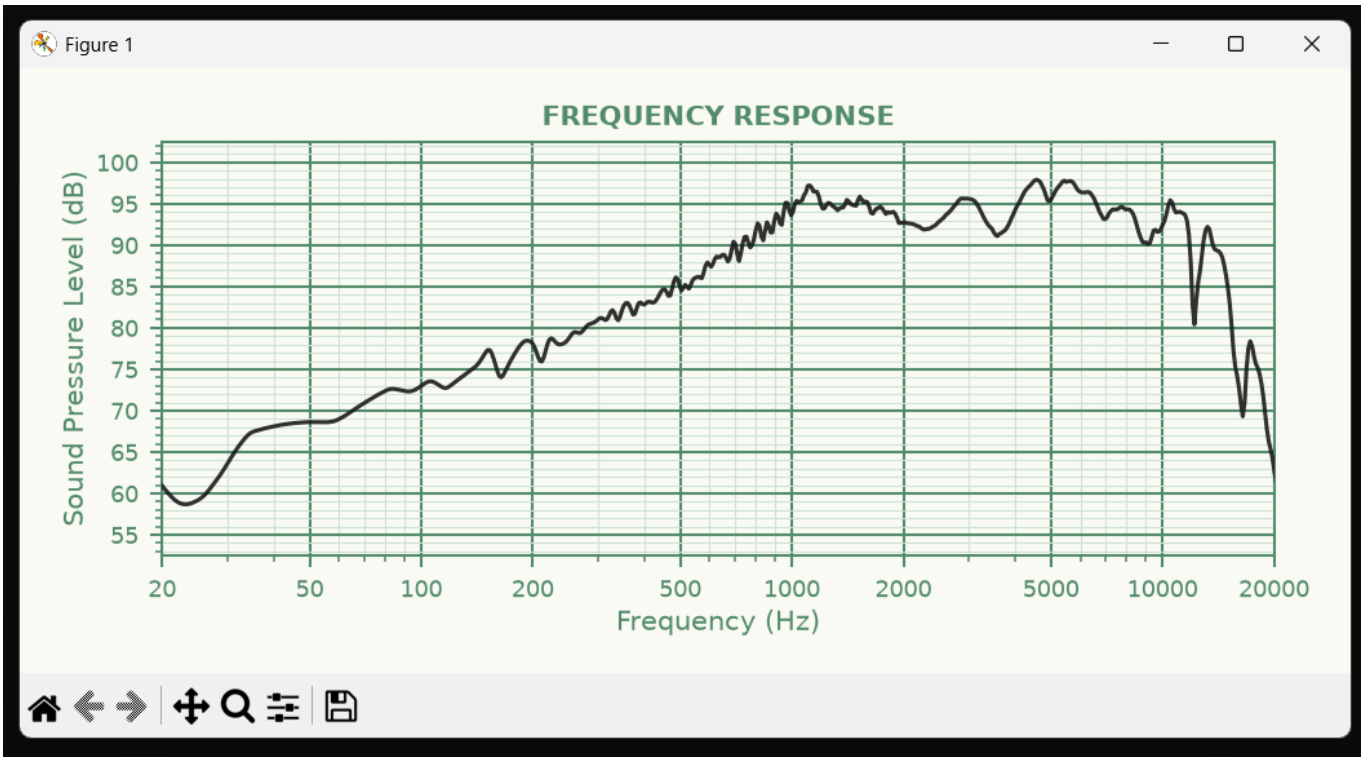
```
C:\...\post_processing_tools>py BandKplot.py mid.frd
```

The data must be provided in three columns: the frequency in Hz, the SPL level in dB, and the phase in degrees. The columns can be whitespace or comma delimited and the data can have any numeric format. The phase data is ignored. Any lines that do not contain three columns or contain other characters are ignored.

To preserve the 20-20k x-axis scale, as well as the 50dB y-axis span, the plot can only be panned up and down to place the data within the plot as you see fit. Zooming is not permitted, and the x-axis range cannot be changed. This presentation/plot was used for loudspeaker systems, and these limits are typical for that sort of data.

The plot is presented in a green scheme by default. The python code can be edited (see line 25) so that the plot is presented with a red scheme.

Example Plot output:



If you wish to have different labels for the title or axis titles, you must edit the python code to change them and then re-run the program. The text for these labels is set on lines 130-132. Feel free to change the text to suit your needs or wishes.

To save the plot to a file, click the floppy disc icon:



To exit, use the window's kill button that is found in the top right corner. It looks like this: ☒.

Groundplane Measurement Post-Processing Tools

These tools are specific for the groundplane measurement techniques described in my online whitepaper “New Approaches To Groundplane Measurements”. This is available from my GitHub repository here:
Whitepaper: New Approaches To Groundplane Measurements

These tools are for data processing only and do not contain any graphical elements or a GUI.

GPDP

GPDP stands for ground plane data processing. This program performs the data processing steps outlined in Appendix of the whitepaper. The input and output files should be in the FRD format. An optional plot shows the resulting frequency response that is produced from the input files by the type of calculation selected by the user.

```
C:\...\post_processing_tools>py GPDP.py –help

usage: use "python GPDP.py --help" for more information

This program performs various corrections to data generated using groundplane-based acoustic measurement
techniques.

options:
  -h, --help            show this help message and exit
  --infile1, --f1 INFILE1
                        Path and Filename of the first input file.
  --infile2, --f2 INFILE2
                        Path and Filename of the second input file.
  --outfile, --of OUTFILE
                        Path and Filename of the output file.
  --calc_type, -c {1,2,3,4}
                        Indicate the type of calculation to perform by number:
                        1 : Calculate the Groundplane Transfer Function as infile1 / infile2 - 1
                        2 : Calculate the Free-Field Response from the Lift and Tuck Data in infile 1 using the
                               Groundplane Transfer Function in infile2
                        3 : Calculate the Reflection Response from the MMMM Measurement 2 Data in infile1 using the
                               Groundplane Transfer Function in infile2
                        4 : Calculate the Free-Field Response from the MMMM Measurement 1 Data in infile1 by
                               subtracting the Reflection Response in infile2
                        NOTE: All input and output files should be in the FRD file format.
  --no-plot
```

When the `--no_plot` option is provided, the program performs the calculation, writes the result to the outfile, and exits.

ImpAlign

When performing the Measure, Move-Move, Measure technique described in the whitepaper, it can be challenging to get the microphone and loudspeaker perfectly aligned in measurements 1 and 2. This will lead to error in the results, because the subtraction of measurement 2 from measurement 1 will not correctly remove the groundplane reflection.
ImpAlign can correct small measurement errors in measurement 2 so that the time domain response perfectly aligns with measurement 1.

Usage:

```
C:\...post_processing_tools>py ImpAlign.py –help

usage: use "python ImpAlign.py --help" for more information

Description:
    This program works with impulse response data generated with the Measure, Move-Move, Measure technique.
    Impulse data from measurement 1 should be supplied in infile1, and from measurement 2 in infile2.
    The program calculates the Cross-Correlation between the infiles and finds the most likely time offset.
    It then optimizes the offset to sub-sample resolution, and recalculates the impulse response supplied in
    infile2 including the offset. The corrected data from infile2 is saved to the outfile. A plot is presented
    showing the optimized time offset. Both infiles must be WAV files with data in the same format (e.g.
    FLOAT32). This format will be use when saving the outfile.

options:
  -h, --help            show this help message and exit
  --infile1, --f1 INFILE1
                        Path and Filename (*.wav) for the Measurement 1 impulse.
  --infile2, --f2 INFILE2
                        Path and Filename (*.wav) for the Measurement 2 impulse.
  --outfile, --of OUTFILE
                        Path and Filename (*.wav) for the aligned file2 data.
  --no-plot
```

This program operates on time domain data provided in WAV files.

The typical use case is after the measurements have been completed but before calculating the frequency domain response. In this case, when using my data collection apps, the H1 impulse file should be used as input, for example the “results_h1.wav” file that is output from DaDooRunRun. Make sure to choose a basename for the files generated by DaDooRunRun so that the data in different files can be distinguished from a different data file.

A plot is generated that shows the time alignment data fit, producing sub-sample accuracy. This can be omitted by using the `--no_plot` option so that the app produces only the corrected measurement 2 WAV file. In this way multiple files can be processed using a script or batch file, without requiring any user interaction. It is recommended to use this app to align the data whenever the Measure, Move-Move, Measure technique is used.